

MiniVSFS: A C-based VSFS Image Generator

You will implement **mkfs_builder**, a C program that creates a raw disk image for a small, inode-based file system **MiniVSFS**. The program takes some parameters from the command line input, and emits a byte-exact image as a binary **.img** file.

You will also implement **mkfs_adder**, another C program that takes a raw MiniVSFS file system image, and a file to be added to that file system. **mkfs_adder** will find the file in the working directory and add it to the root directory (/) of the file system. Finally, it will save the output image in the new file specified by the output flag.

To understand how to start working on the project, refer to the **Project Starting Point** section.

MiniVSFS

MiniVSFS, based on VSFS, is fairly simple – a block-based file system structure with a superblock, inode and data bitmaps, inode tables, and data blocks. Compared to the regular VSFS, MiniVSFS cuts a few corners:

- Indirect pointer mechanism is not implemented
- Only supported directory is the root (/) directory
- Only one block each for the inode and data bitmap
- Limited size and inode counts

What You'll Build

MKFS_BUILDER

```
mkfs_builder \  
  --image out.img \  
  --size-kib <180..4096> \  
  --inodes <128..512>
```

Parameter	Description
-----------	-------------

image	the name of the output image
size-kib	the total size of the image in kilobytes (multiple of 4)
inodes	number of inodes in the file system

MKFS_ADDER

```
mkfs_adder \
--input out.img
--output out2.img
--file <file>
```

Parameter	Description
input	the name of the input image
output	name of the output image
file	the file to be added to the file system

Output

The updated output binary image with the file added.

Program Workflow

1. MKFS_BUILDER

It should perform the following tasks in order:

1. Parse the command line inputs
2. Create the file system according to the provided specifications
3. Save the file system as a binary file with the name specified by the `--image` flag

2. MKFS_ADDER

It should perform the following tasks in order:

1. Parse the command line inputs
2. Open the input image as a binary file
3. Search the file in the present working directory, and add the file to the file system
4. Update the file system binary image

MiniVSFS Specifications

Parameter	Value
Block Size	4096 Bytes
Inode Size	128 Bytes
Total Blocks	$\text{size_kib} \times 1024 / 4096$

Disk Model

All on-disk structures are little endian in format. Disks are divided into equally sized (4096 B) blocks, arranged as follows:

1. Superblock (1 block)
2. Inode Bitmap (1 block)
3. Data Bitmap (1 block)
4. Inode Table
5. Data

Superblock

The superblock is placed on the **first block (block 0)** of the image.

Field	Size (bytes)	Default
magic	4	0x4D565346
version	4	1
block_size	4	4096
total_blocks	8	
inode_count	8	
inode_bitmap_start	8	
inode_bitmap_blocks	8	
data_bitmap_start	8	
data_bitmap_blocks	8	
inode_table_start	8	
inode_table_blocks	8	
data_region_start	8	
data_region_blocks	8	
root_inode	8	1
mtime_epoch	8	Build time (Unix Epoch)
flags	4	0
checksum	4	Check discussion on checksum

Skeleton for the superblock has been created as the struct `superblock_t`.

Inodes

Inodes are **128-byte structures**.

Field	Size (bytes)	Default
mode	2	Check discussion on modes
links	2	Check discussion on links
uid	4	0
gid	4	0
size_bytes	8	
atime	8	Build time (Unix Epoch)
mtime	8	Build time (Unix Epoch)
ctime	8	Build time (Unix Epoch)
direct[12]	4 each	
reserved_0	4	0
reserved_1	4	0
reserved_2	4	0
proj_id	4	Your group ID
uid16_gid16	4	0
xattr_ptr	8	0
inode_crc	8	Check discussion on checksum

Twelve direct blocks are allowed in MiniVSFS. Elements inside the direct array are absolute data block numbers. Skeleton for the inode has been created as [inode_t](#).

Note: Inodes do not have an explicit id/inode number field. They are referred to by their index in the inode table.

The [root_inode](#) field in the superblock struct is set as **1**, as inodes are **1-indexed** in the table. For unused direct blocks, set them to **0**.

Inode Modes

Type	Mode
File	(0100000) ₈
Directory	(0040000) ₈

Bitmaps (Inode and Data)

Rule	Description
Bit = 1	Allocated
Bit = 0	Free
Bit 0 of byte 0	First object (inode #1 or first data block)
Bitmap size	Occupies entire block (zero padded tail)

Directory Entry

A directory entry stores information about parent and child files.

Field	Size (bytes)	Default
inode_no	4	0 if free
type	1	1 = file, 2 = dir
name	58	
checksum	1	Check discussion on checksum

Skeleton for this entry has been created as `dirent64_t`.

File Allocation Policy

- Inodes and data blocks are placed using **first-fit allocation**.
- The first available block is allotted to the new resource.
- If a file is too large to be accommodated with **12 direct blocks**, return a warning message.

Root Directory

The root directory:

- Has a fixed inode number of **1**
- Contains two members:
 - .
 - ..

Both point to itself.

The root directory occupies the **first data block** to store its entries.

Links

The **links field** of the inode contains the number of directories pointing to it.

Case	Link Count
Root directory initially	2 (. and ..)
Each file inside root	1

When a new file is created:

- The root link count increases by **1**
- The new file refers to root through ..

Checksum

Each structure requires a checksum:

- superblock
- inode
- directory entry

Use the provided functions:

```
superblock_crc_finalize(&superblock);  
inode_crc_finalize(&inode);  
dirent_checksum_finalize(&dirent);
```

Example:

```
superblock_t superblock;  
inode_t inode;  
dirent64_t dirent;  
  
/* proper configuration of the structures */  
  
superblock_crc_finalize(&superblock);  
inode_crc_finalize(&inode);  
dirent_checksum_finalize(&dirent);
```

Time

Unix epoch time can be obtained using the C [time](#) library.

```
#include <time.h>
```

```
time_t now = time(NULL);
```

Discussion

- Compilation errors may occur until the required structures are filled correctly.
- Since the project involves binary data, understanding **pointers and typecasting** is essential.
- Each metadata structure should correspond to a **C struct** for organization.
- Suggested data types:

Size	Type
8-bit	uint8_t
16-bit	uint16_t
32-bit	uint32_t
64-bit	uint64_t

- Proper **error handling** should be implemented to avoid segmentation faults.
- The provided skeleton `.c` files contain comments to help guide implementation.
- To debug the binary output file, tools such as:

hexdump
xxd

can be used.

Project Starting Point

You will receive the following items in a compressed `.zip` file:

- `mkfs_builder.c` skeleton
- `mkfs_adder.c` skeleton
- Files to be added to MiniVSFS

Steps:

1. Download the assigned `.zip` file
2. Extract the files
3. Start working on the skeleton files

Project Deliverables

Item

Complete `mkfs_builder.c`

Complete `mkfs_adder.c`

Note: You do **not** need to submit the sample files provided.